PROJET DE FIN DE MODULE CLOUD ET INFRASTRUCTURES VIRTUELLES

SOC Lab Conteneurisé

Déploiement et exploitation d'un centre opérationnel de sécurité

Wazuh · Suricata · Grafana · TheHive · Docker Compose

Formation	Module	Notation
L2 SIMAC P11	**Cloud et Infrastructures Virtuelles**	**/ 40 points**

Informations étudiant

Champ	À remplir
Nom & Prénom	
Numéro étudiant	
Groupe	
Date de rendu	
Dépôt Git	

Objectif du projet

Déployer, configurer et exploiter un SOC (Security Operations Center) complet en utilisant Docker Compose. Le projet évalue votre maîtrise de la conteneurisation appliquée à la cybersécurité : architecture multi-services, persistance des données, réseaux isolés, et réponse à incident.

Ce projet est à réaliser individuellement ou en binôme. Chaque livrable doit être versionné dans un dépôt Git privé partagé avec l'enseignant avant la date limite.

Durée totale

Phase
Mise en place infrastructure

Modalités de rendu

- Dépôt Git structuré
- Rapport PDF (6-10 pages)

Implémentation et configuration	
Tests et rapport	
Total estimé	

🎥 Démo live ou vidéo (5 min)

💻 Stack fonctionnelle au moment de la soutenance

> ⚠ **Rendu incomplet**
> Un rapport sans stack fonctionnelle est noté au maximum 20/40.

Partie 1 — Infrastructure Docker

Architecture, réseaux, volumes, secrets

14 points

Objectif de la partie

Vous devez déployer l'intégralité du SOC Lab en utilisant Docker Compose. Chaque service doit être correctement configuré, les données persistées entre redémarrages, les secrets externalisés dans .env, et les réseaux segmentés selon le principe du moindre privilège.

L. 1 docker-compose.yml — Stack SOC complète

~3h

Rédigez le fichier docker-compose.yml qui orchestre les 5 services du SOC Lab. Ce fichier est le cœur de votre projet — il doit être propre, commenté, et fonctionnel en une seule commande.

Service	Image à utiliser	Ports à exposer	Contrainte particulière
wazuh-manager	wazuh/wazuh-manager:4.7	1514/udp, 1515/tcp, 55000/tcp	healthcheck obligatoire
wazuh-dashboard	wazuh/wazuh-dashboard:4.7	443:5601	depends_on wazuh-manager
suricata	jasonish/suricata:latest	(pas de port exposé)	network_mode: host ou cap_add
grafana	grafana/grafana:10.2	3000:3000	volume pour les dashboards
thehive	strangebee/thehive:5	9000:9000	dépend de la BDD Cassandra

Exigences techniques obligatoires :

1. Tous les services doivent être sur le réseau soc-network
2. Suricata doit avoir accès à l'interface réseau réelle de l'hôte (network_mode: host ou CAP_NET_RAW)
3. Les données Wazuh, Grafana et TheHive doivent persister via des volumes nommés
4. Wazuh Manager doit avoir un healthcheck fonctionnel (test via curl ou wget)
5. Aucun mot de passe ni secret en dur dans le YAML — utiliser ${VARIABLE} depuis .env

⚠ Contrainte vm.max_map_count

Wazuh Indexer (ElasticSearch) requiert vm.max_map_count=262144. Votre rapport doit expliquer pourquoi cette valeur système est nécessaire et comment la rendre persistante sur l'hôte.

L. 2

Fichier .env et segmentation réseau

~1h

La sécurisation des secrets et la segmentation réseau sont des exigences de production. Ce livrable évalue votre compréhension des bonnes pratiques DevSecOps.

1. Créez .env avec toutes les variables sensibles (mots de passe, clés API, ports). Créez .env.example avec des valeurs fictives.
2. Créez .gitignore protégeant .env tout en permettant le commit de .env.example.
3. Segmentez les réseaux : créez au minimum soc-frontend (dashboard↔extérieur) et soc-backend (services internes). Justifiez votre choix dans le rapport.
4. Vérifiez l'isolation avec docker network inspect et la matrice de connectivité ci-dessous.

Matrice de connectivité à valider et documenter dans votre rapport :

Source → Destination	Résultat attendu	Commande de test	Votre résultat
grafana → wazuh-manager	✓ Joignable	docker exec grafana wget -qO- http://wazuh-manager:55000	
suricata → grafana	✗ Injoignable	docker exec suricata nslookup grafana	
thehive → wazuh-manager	✓ Joignable	docker exec thehive nslookup wazuh-manager	
dashboard → suricata	✗ Injoignable	docker exec wazuh-dashboard nslookup suricata	

Barème — Partie 1

Critère	Max	Obtenu	Commentaire
docker-compose.yml complet et fonctionnel (docker compose up -d) Les 5 services démarrent sans erreur et atteignent l'état "healthy"			
Volumes nommés correctement définis et persistants après restart			
Healthcheck fonctionnel sur wazuh-manager			
Fichier .env complet + .env.example + .gitignore correct			
Segmentation réseau justifiée + matrice de connectivité validée			
TOTAL PARTIE 1	**14**		

Partie 2 — Simulation d'attaque et détection

Suricata · Wazuh · Réponse à incident

16 points

Objectif de la partie

Vous allez simuler les 3 phases d'une attaque APT (Advanced Persistent Threat) contre votre SOC Lab, observer la détection par Suricata et Wazuh, puis gérer l'incident dans TheHive. Chaque phase doit être documentée avec des captures d'écran.

Usage légal uniquement

Toutes les attaques simulées doivent être effectuées dans l'environnement Docker isolé, uniquement sur vos propres conteneurs. Toute utilisation de ces techniques sur un réseau ou système tiers est illégale

L. 3 Phase 1 — Reconnaissance réseau (Nmap)

~1h

La reconnaissance est la première phase du Kill Chain de Lockheed Martin. L'attaquant cartographie les services disponibles avant de choisir son vecteur d'attaque.

1. Lancez un scan Nmap SYN (-sS) sur l'adresse IP de votre hôte Docker. Ciblez les ports 1-10000.
2. Observez les alertes générées dans le Wazuh Dashboard (Security Events). Notez : le niveau d'alerte, la règle déclenchée, le timestamp.
3. Consultez les logs Suricata en direct et filtrez uniquement les alertes de type "scan".
4. Documentez avec une capture d'écran de l'alerte dans Wazuh Dashboard.

Résultats attendus à documenter :

Élément observé	Valeur attendue	Votre observation
Règle Wazuh déclenchée	86602 (Network Scan)	
Niveau de sévérité	10 (MEDIUM)	
Signature Suricata	ET SCAN Nmap SYN Scan	
Délai de détection	< 30 secondes	

L. 4 Phase 2 — Brute Force SSH + Réponse Wazuh

~1h

Le brute force SSH est l'une des attaques les plus courantes. Wazuh intègre un mécanisme de réponse active qui peut bloquer automatiquement l'IP attaquante via iptables.

1. Activez un service SSH dans un conteneur de test (image ubuntu:22.04). Configurez-le avec un mot de passe faible.
2. Lancez Hydra avec la liste rockyou.txt (10 premiers mots de passe suffisent pour déclencher l'alerte).
3. Observez dans Wazuh : combien de tentatives déclenchent l'alerte level 10 ? Quelle règle est activée ?
4. Configurez la réponse active Wazuh pour bloquer automatiquement l'IP après 5 tentatives. Testez que le blocage fonctionne.

Questions d'analyse à répondre dans le rapport :

5. Quel est le seuil par défaut de Wazuh avant de déclencher l'alerte brute force ? Comment le modifier ?
6. La réponse active Wazuh utilise iptables. Quelle est la durée de blocage par défaut ? Comment la rendre permanente ?
7. Quelle différence entre les alertes level 5, 10 et 15 dans Wazuh ? Citez un exemple de chaque dans vos logs.

L. 5 Phase 3 — Injection SQL + Gestion de crise TheHive

~1h30

La phase finale combine une attaque web et la réponse à incident complète. Vous allez détecter une injection SQL via Suricata, puis gérer le cycle de vie de l'incident dans TheHive selon le standard PICERL.

1. Ajoutez un conteneur DVWA (Damn Vulnerable Web Application) dans votre stack. Configurez-le sur le port 8080.
2. Exécutez une injection SQL manuelle (UNION SELECT) sur le formulaire de login de DVWA. Vérifiez que Suricata détecte la signature.
3. Ouvrez un cas dans TheHive avec toutes les informations : titre, sévérité HIGH, TLP:AMBER, PAP:AMBER.
4. Créez les 4 tâches du cycle PICERL dans TheHive et documentez les observables (IP source, URL, payload SQL).
5. Appliquez les mesures de containment Docker (déconnexion réseau du conteneur compromis).

Structure du cas TheHive à créer :

Champ TheHive	Valeur à renseigner
Titre	APT-2024-XXX — SQLi détectée sur DVWA — TechCorp SA
Sévérité	HIGH
TLP	AMBER (partage restreint à l'organisation)
PAP	AMBER (enrichissement limité)
Tâche 1	Identification — Analyser les logs Suricata et Wazuh
Tâche 2	Containment — Isoler le conteneur DVWA du réseau
Tâche 3	Éradication — Patcher la vulnérabilité SQLi
Tâche 4	Recovery — Vérifier l'intégrité et remettre en service
Observable 1	IP source de l'attaquant (type: IP)
Observable 2	URL avec le payload SQL (type: URL)

Barème — Partie 2

Critère	Max	Obtenu	Commentaire
Scan Nmap détecté dans Wazuh + capture d'écran alerte			
Brute Force SSH déclenche alerte level 10 dans Wazuh			
Réponse active configurée et blocage IP opérationnel			
Injection SQL détectée par Suricata (capture EVE JSON)			
Cas TheHive complet : titre, sévérité, TLP, 4 tâches, 2 observables			
Containment Docker appliqué et vérifié (isolation réseau)			
TOTAL PARTIE 2	**16**		

Partie 3 — Rapport technique

Documentation, analyse, recommandations

10 points

Objectif de la partie

Un projet technique sans documentation est un projet incomplet. Cette partie évalue votre capacité à rédiger un rapport professionnel, à analyser les résultats obtenus, et à formuler des recommandations de sécurité pertinentes.

L. 6 Rapport PDF — Structure imposée

~2h

Le rapport doit être rédigé en PDF (6 à 10 pages, hors annexes). Il suit la structure imposée ci-dessous. La qualité de rédaction et la rigueur technique sont évaluées.

Section	Contenu attendu	Pages recommandées
1. Introduction	Contexte, objectifs, présentation de TechCorp SA	0.5
2. Architecture	Schéma de l'architecture Docker (dessiné ou généré), justification des choix	1
3. Déploiement	Difficultés rencontrées, solutions appliquées, extraits de configuration clés	1.5
4. Résultats des attaques	Timeline des 3 phases, captures d'écran annotées, analyse des alertes	2
5. Analyse de sécurité	Points forts et points faibles détectés, faux positifs éventuels	1
6. Recommandations	3 recommandations concrètes pour durcir le SOC en production	1
7. Conclusion	Bilan des compétences acquises, extensions envisageables	0.5
Annexes	Fichiers de config complets, logs bruts, commandes utilisées	variable

Schéma d'architecture imposé :

Le schéma doit représenter : les 5 conteneurs, les réseaux Docker (soc-frontend, soc-backend), les volumes nommés, et les flux de données entre services. Vous pouvez utiliser draw.io, Mermaid, ou tout outil de votre choix.

Les 3 recommandations de sécurité doivent couvrir :

1. Une recommandation sur les secrets et la gestion des credentials (ex: Vault, Docker Secrets)
2. Une recommandation sur le durcissement des conteneurs (ex: read-only filesystem, non-root user, seccomp)

3. Une recommandation sur la supervision et les alertes (ex: notification Slack/email, escalade automatique)

Barème — Partie 3

Critère	Max	Obtenu	Commentaire
Structure complète du rapport (7 sections présentes)			
Schéma d'architecture lisible et correct			
Analyse des alertes : timeline + captures annotées			
3 recommandations de sécurité pertinentes et concrètes			
Qualité rédactionnelle, clarté, références techniques			
TOTAL PARTIE 3	**10**		

Récapitulatif et notation finale

40 points

Partie	Livrables	Barème	Note	%
Partie 1 — Infrastructure Docker	L1 + L2	**/14**		35%
Partie 2 — Simulation d'attaque	L3 + L4 + L5	**/16**		40%
Partie 3 — Rapport technique	L6	**/10**		25%
TOTAL GÉNÉRAL		**/ 40**		

Pénalités

−5 pts : dépôt Git absent ou non structuré

−3 pts : secrets en dur dans docker-compose.yml

−3 pts : stack non fonctionnelle au rendu

−2 pts : rapport hors délai (par jour de retard)

Bonus (max +4 pts)

+2 pts : intégration MISP (threat intelligence)

+1 pt : notifications Slack/email configurées

+1 pt : déploiement réussi sur VM cloud (AWS/OVH)

Structure du dépôt Git attendue

Structure Git imposée

```
soc-lab-[NOM_ETUDIANT]/
├── README.md            ← Instructions de déploiement (obligatoire)
├── docker-compose.yml
├── .env.example         ← Valeurs fictives — JAMAIS le vrai .env
├── .gitignore           ← .env ignoré
├── wazuh/
│   └── config/
├── suricata/
│   ├── rules/           ← Règles personnalisées
│   └── logs/            ← .gitkeep (répertoire vide)
├── grafana/
│   └── dashboards/
├── thehive/
│   └── config/
└── rapport/
    ├── rapport-final.pdf
    └── captures/        ← Toutes les captures d'écran
```

⚠ Commit du .env interdit

Tout dépôt contenant un fichier .env avec de vrais mots de passe sera pénalisé de −5 pts, même si le fichier est supprimé dans un commit ultérieur (l'historique Git est permanent).

Ressources et documentation

Ressource	URL / Commande
Documentation Wazuh	https://documentation.wazuh.com/current/
Suricata User Guide	https://suricata.readthedocs.io/
TheHive Project	https://docs.strangebee.com/thehive/
Règles Suricata (ET Open)	https://rules.emergingthreats.net/open/
Docker Compose référence	https://docs.docker.com/compose/compose-file/
Aide-mémoire commandes SOC	voir Section 6 du document de référence

✓ Conseil final

Commencez par démarrer la stack avec docker compose up -d et vérifiez que les 5 services atteignent l'état "healthy" avant de passer aux phases d'attaque. Une infrastructure stable est la base de tout. Documentez chaque étape au fur et à mesure — ne laissez pas le rapport pour la fin.